```
from google.colab import files
files.upload()
```

Choose Files diabetes.csv

diabetes.csv(text/csv) - 23873 bytes, last modified: 2/2/2026 - 100% done

```
Saving diabetes.csv to diabetes.csv
{'diabetes.csv':
b'Pregnancies,Glucose,BloodPressure,SkinThickness,Insulin,BMI,DiabetesPedigreeFunction,Age,Outcome\r\n6,148,72,35,0,33.6
```

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import seaborn as sns
import matplotlib.pyplot as plt


df = pd.read_csv('diabetes.csv')

df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Next steps: Generate code with df New interactive sheet

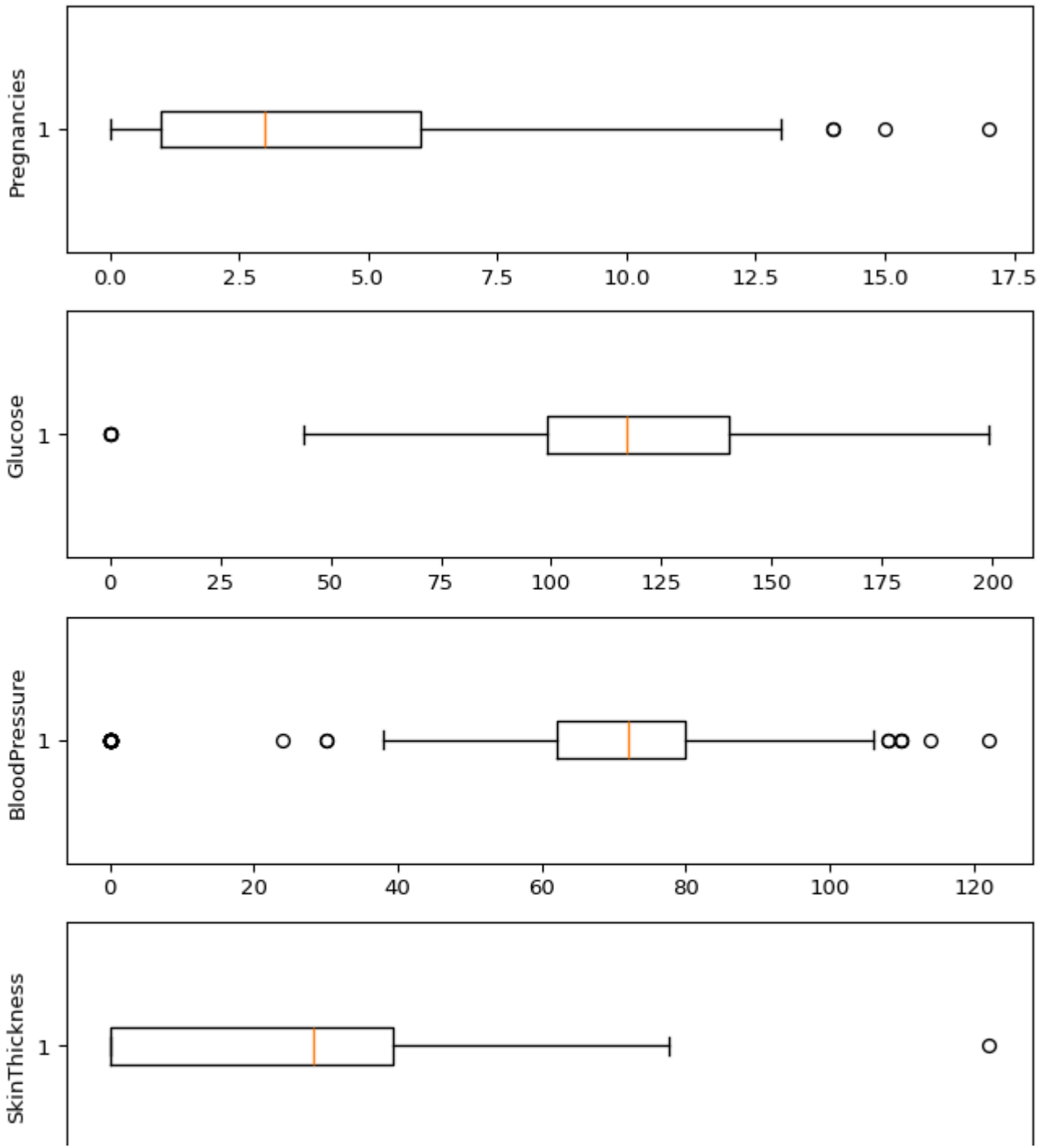
```
df.info()


print(df.isnull().sum())
```

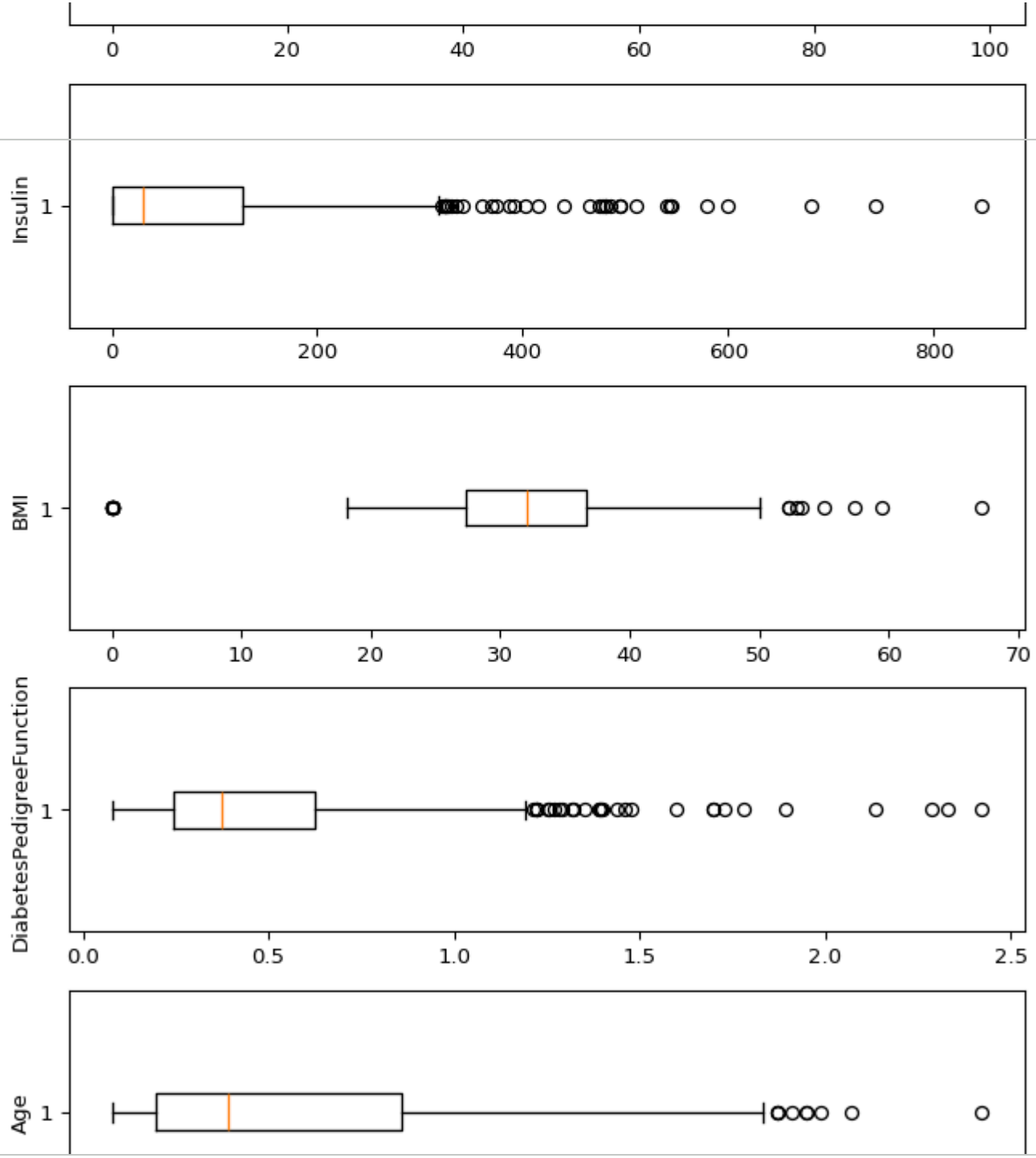
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

```
df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000

```
fig, axs = plt.subplots(len(df.columns), 1, figsize=(7, 18), dpi=95)

for i, col in enumerate(df.columns):
    axs[i].boxplot(df[col], vert=False)
    axs[i].set_ylabel(col)

plt.tight_layout()
plt.show()
```

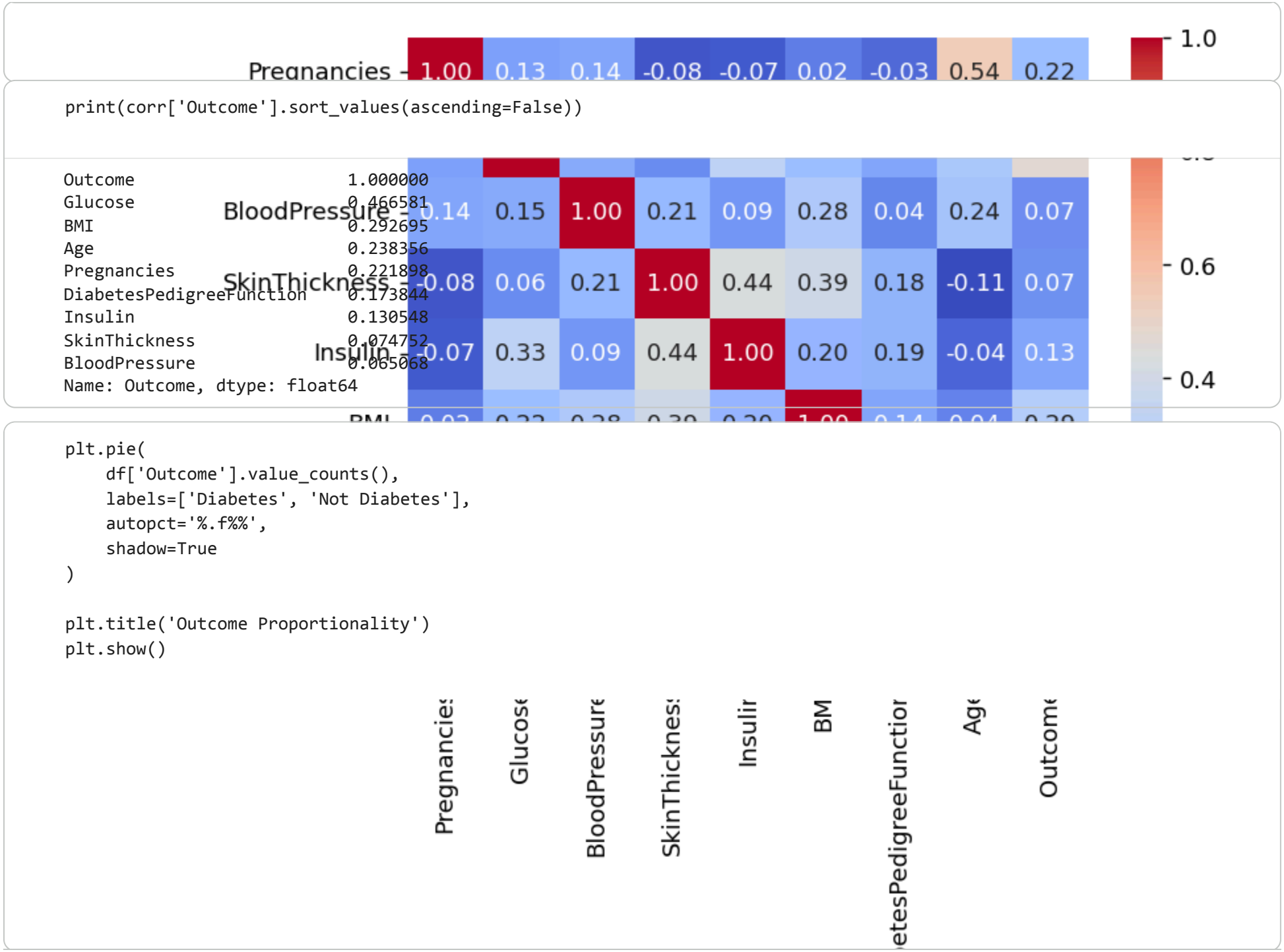

Pregnancies
1
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Glucose
1
0
25
50
75
100
125
150
175
200
BloodPressure
1
0
20
40
60
80
100
120
SkinThickness
1


0
20
40
60
80
100
Insulin
1
0
200
400
600
800
BMI
1
0
10
20
30
40
50
60
70
DiabetesPedigreeFunction
1
0.0
0.5
1.0
1.5
2.0
2.5
Age
1

```
q1, q3 = np.percentile(df['Insulin'], [25, 75])
iqr = q3 - q1

lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr

clean_df = df[(df['Insulin'] >= lower) & (df['Insulin'] <= upper)]

print("Original shape:", df.shape)
print("After removing outliers:", clean_df.shape)
```

```
Original shape: (768, 9)
After removing outliers: (734, 9)
```

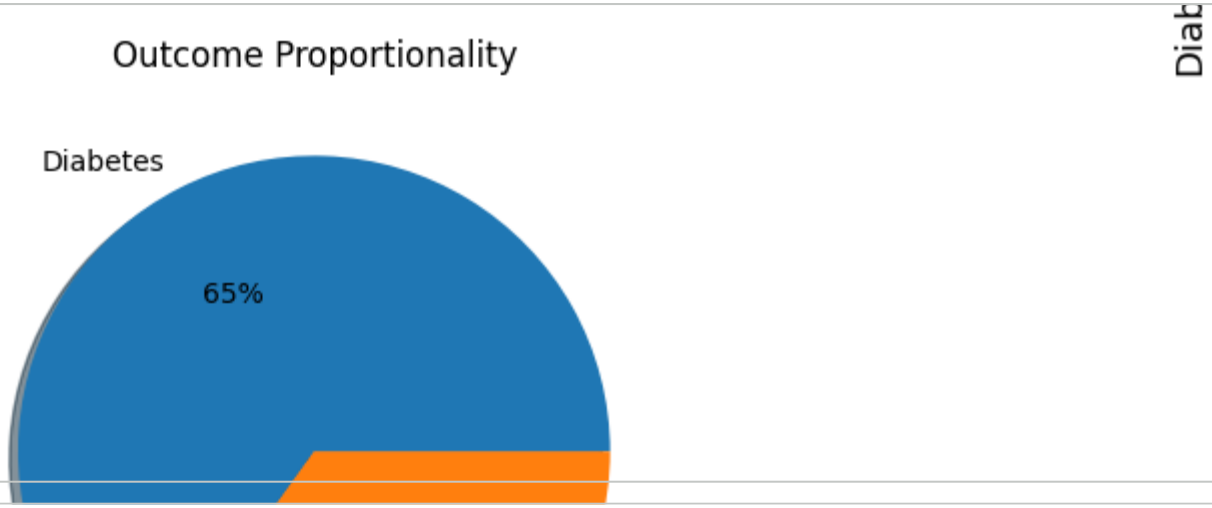
```
corr = df.corr()

plt.figure(dpi=130)
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
plt.show()
```




```
print(corr['Outcome'].sort_values(ascending=False))
```

```
Outcome                    1.000000
Glucose                    0.466581
BMI                        0.292695
Age                        0.238356
Pregnancies                0.221898
DiabetesPedigreeFunction   0.173844
Insulin                    0.130548
SkinThickness              0.074752
BloodPressure              0.065068
Name: Outcome, dtype: float64
```

```
plt.pie(
    df['Outcome'].value_counts(),
    labels=['Diabetes', 'Not Diabetes'],
    autopct='%.f%%',
    shadow=True
)

plt.title('Outcome Proportionality')
plt.show()
```




```
X = df.drop(columns=['Outcome'])
y = df['Outcome']

print("Features shape:", X.shape)
print("Target shape:", y.shape)
```

```
Features shape: (768, 8)
Target shape: (768,)
```

```
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)

print(X_normalized[:5])
```